

第八章：进程间通信（IPC）

一、核心概念

1.1 IPC 的必要性

进程之间的地址空间是隔离的——进程 A 不能直接读写进程 B 的内存。当进程需要协作时，必须通过操作系统提供的 IPC 机制来交换数据和同步状态。

1.2 IPC 机制分类

按数据传输方式分类：

类别	机制	数据模型	特点
字节流	管道 (pipe)	无边界的字节流	FIFO，单向，父子进程
消息传递	消息队列 (msgqueue)	带类型的消息	有边界，可按类型筛选
共享内存	shm	共享物理页	最快，需自行同步
事件通知	eventfd	64 位计数器	轻量，适合通知
异步信号	signal	无数据载荷	异步，打断执行流

按 API 风格分类：

风格	代表	命名方式	持久性
System V IPC	msgget/msgsnd/shmget/shmat	整数 key	内核持久
POSIX IPC	mq_open/shm_open	文件名	文件系统持久

1.3 管道（Pipe）

管道是最经典的 IPC 机制：

- **匿名管道**：`pipe(fds)` 创建，只能在有亲缘关系的进程间使用
- **有名管道（FIFO）**：通过文件系统路径访问，任意进程可用
- 特性：FIFO 字节流、固定大小缓冲区、写端关闭后读端读到 EOF

1.4 共享内存

最高效的 IPC——数据不需要拷贝，多个进程直接访问同一块物理内存。但由于没有内置同步机制，需要配合信号量/futex/spinlock 使用。

1.5 信号

信号是唯一的异步 IPC 机制——接收方不需要主动调用 `read` 或 `recv`，内核在返回用户态时自动投递。详见第二章（进程管理）和第五章（中断与异常）。

二、基本推论

推论 1: IPC 存在"复制次数"的根本权衡。 管道/消息队列需要至少两次数据拷贝（发送方→内核缓冲区→接收方）。共享内存零拷贝，但需要额外的同步机制。

推论 2: 管道的生产者-消费者模型天然适合 shell 管线。 `ls | grep foo | wc -l` 中每个 `|` 就是一个管道，前一个命令的 stdout 连接后一个命令的 stdin。

推论 3: eventfd 比 pipe 更适合纯事件通知。 pipe 传输字节流有缓冲区拷贝开销，eventfd 只有一个原子计数器，read/write 操作是简单的加减。

推论 4: 共享内存 + futex 是用户态高性能同步的标准方案。 共享内存提供数据通道，futex 在无竞争时纯用户态操作，有竞争时才陷入内核等待。

三、Linux / 通用实现

3.1 Linux 管道

```
struct pipe_inode_info {
    struct mutex mutex;
    wait_queue_head_t rd_wait, wr_wait;
    unsigned int head, tail;
    struct pipe_buffer *bufs; // 环形缓冲区（默认 16 页 = 64KB）
    unsigned int ring_size;
};
```

- 默认缓冲区 64KB (`F_SETPIPE_SZ` 可调)
- 读写用 `wait_queue` 阻塞/唤醒
- `splice()/vmsplice()` 可以零拷贝传输

3.2 Linux System V 消息队列

```
struct msg_queue {
    struct kern_ipc_perm q_perm;
    time64_t q_stime, q_rtime, q_ctime;
    unsigned long q_cbytes; // 当前总字节数
    unsigned long q_qnum; // 当前消息数
    unsigned long q_qbytes; // 最大字节数
    struct list_head q_messages; // 消息链表
    struct list_head q_receivers, q_senders; // 等待者
};
```

- `msgsnd()`: 分配 `msg_msg`, 入队, wakeup 接收者
- `msgrcv()`: 按 type 筛选, 出队, wakeup 发送者

3.3 Linux 共享内存

System V: `shmget()` → `shmat()` → 使用 → `shmdt()` → `shmctl(IPC_RMID)`

POSIX: `shm_open()` → `mmap()` → 使用 → `munmap()` → `shm_unlink()`

底层都是通过 `mmap()` 将同一物理页映射到多个进程的虚拟地址空间。

3.4 Linux eventfd

```
struct eventfd_ctx {
    struct kref kref;
    wait_queue_head_t wqh;
    __u64 count;
    unsigned int flags;
};
```

- `write(fd, &val)`: `count += val`; 如果溢出到 `UINT64_MAX` 则阻塞
- `read(fd, &buf)`: 普通模式 `buf = count`, `count = 0`; 信号量模式 `buf = 1`, `count--`
- 常用于 `epoll` 事件通知和虚拟化 (KVM 使用 `eventfd` 做中断注入)

四、RUOS 的实现

核心文件

- `src/fs/pipe.c` — 管道
- `src/ipc/msgqueue.h` — 消息队列
- `src/mm/shm.h` — 共享内存
- `src/fs/eventfd.c` — `eventfd`
- `src/proc/signal.c` — 信号

4.1 pipe

```
struct pipe {
    struct spinlock lock;
    char data[PIPE_SIZE]; // 环形缓冲区 (512 字节)
    uint nread, nwrite; // 累计读/写字节数
    int readopen, writeopen;
};
```

- 可读数据量 = `nwrite - nread`
- 缓冲区满 = `nwrite - nread == PIPE_SIZE`
- 写满时 `sleep(&pi->nwrite, &pi->lock)`, 读空时 `sleep(&pi->nread, &pi->lock)`

对比 Linux: Linux 管道 64KB 缓冲区, RUOS 只有 512 字节。Linux 用 `wait_queue`, RUOS 用 `sleep channel`。

4.2 消息队列

```

#define MAX_MSG_QUEUES 64
#define MAX_MSGS 32
#define MAX_MSG_SIZE 4096

struct msg_queue {
    int key;
    struct msg_entry *head, *tail;
    int msgcount, bytes_used;
    void *send_chan, *recv_chan; // sleep channel
};

```

type 筛选规则:

- `type == 0`: 取第一条 (不管类型)
- `type > 0`: 取第一条类型匹配的
- `type < 0`: 取类型 $\leq |type|$ 的最小类型消息

4.3 共享内存

```

struct shm_seg {
    int key;
    void *kaddr; // 内核虚拟地址
    uint64 size;
    int refcount;
    struct shmid_ds shmid_ds;
};

```

`shmat()` 将 `kaddr` 的物理页映射到用户页表。进程的 `shm_attach[16]` 记录已 attach 的段。

对比 Linux: RUOS 的 shm 不支持 fork 自动继承 (需重新 `shmat`)，不支持 POSIX `shm_open`。但 System V 语义完整。

4.4 eventfd (决赛题目)

```

struct eventfd {
    struct spinlock lock;
    uint64 counter;
    int flags; // EFD_NONBLOCK | EFD_SEMAPHORE | EFD_CLOEXEC
    int refcount;
};

```

三种模式:

```

普通:      write(+7), write(+3) → read = 10, counter = 0
信号量:    write(+3) → read=1,c=2; read=1,c=1; read=1,c=0
非阻塞:    counter=0 时 read → -EAGAIN

```

集成为 `struct file` (type=FD_EVENTFD), dup/fork 共享同一个 eventfd 对象。

4.5 各机制选择指南

需要字节流?

→ pipe

需要消息边界?

→ msgqueue

需要高吞吐零拷贝?

→ shm + futex/spinlock

只需事件通知?

→ eventfd

需要异步打断?

→ signal

五、八股 / 面试高频问题

- Q: Linux 有哪些 IPC 机制? 各自优缺点?** A: 管道（简单但单向、需亲缘关系）、消息队列（有边界但有大小限制）、共享内存（最快但需同步）、信号（异步但信息量少）、Socket（最灵活、可跨主机但开销大）、eventfd（轻量事件通知）。
- Q: 共享内存为什么最快?** A: 其他 IPC 都需要至少两次拷贝（用户→内核→用户），共享内存是多个进程映射同一物理页，数据直接在内存中可见，零拷贝。代价是需要额外的同步机制。
- Q: 管道和消息队列的区别?** A: 管道是字节流（无边界），FIFO 有序；消息队列有消息边界和类型，可按类型筛选接收。管道通常用于父子进程的简单通信，消息队列适合多生产者多消费者。
- Q: eventfd 和 pipe 的区别? 使用场景?** A: eventfd 只有一个 64 位计数器，无缓冲区拷贝，适合纯事件通知和计数。pipe 是字节流，可传输任意数据。eventfd 常用于 epoll 事件循环（通知 epoll_wait 退出）和 KVM 中断注入。
- Q: 什么是 POSIX 信号量? 和 System V 信号量的区别?** A: POSIX 信号量基于文件系统命名（sem_open），接口更简洁。System V 信号量基于 key/id，支持信号量集（多个信号量原子操作）。POSIX 信号量是现代推荐。

六、项目贡献亮点

- 实现了 pipe/msgqueue/shm/eventfd/signal 五种 IPC 机制，覆盖字节流到事件通知的完整谱系
- eventfd 是决赛现场实现的核心题目，支持普通/信号量/非阻塞三种模式
- 理解不同 IPC 机制的设计取舍和适用场景

七、设计取舍总结

决策	RUOS 选择	Linux 做法	权衡
管道缓冲区	512B	64KB (16页)	简单但吞吐低
消息队列	自实现 System V	内核 IPC 子系统	完整但简化
共享内存	System V shmget/shmat	System V + POSIX	只实现一种

决策	RUOS 选择	Linux 做法	权衡
eventfd	完整实现（3种模式）	同 + epoll 集成	无 epoll 支持
同步辅助	sleep/wakeup	wait_queue + futex	简单但通用性低